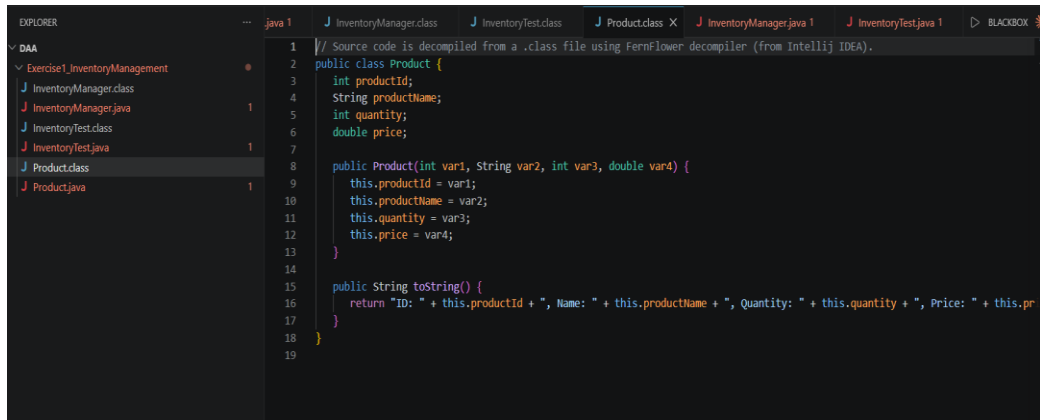
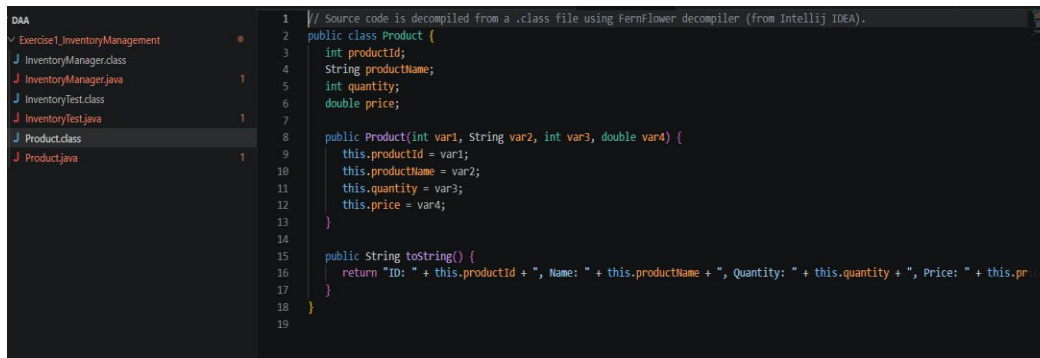


Exercise 1: Inventory Management System

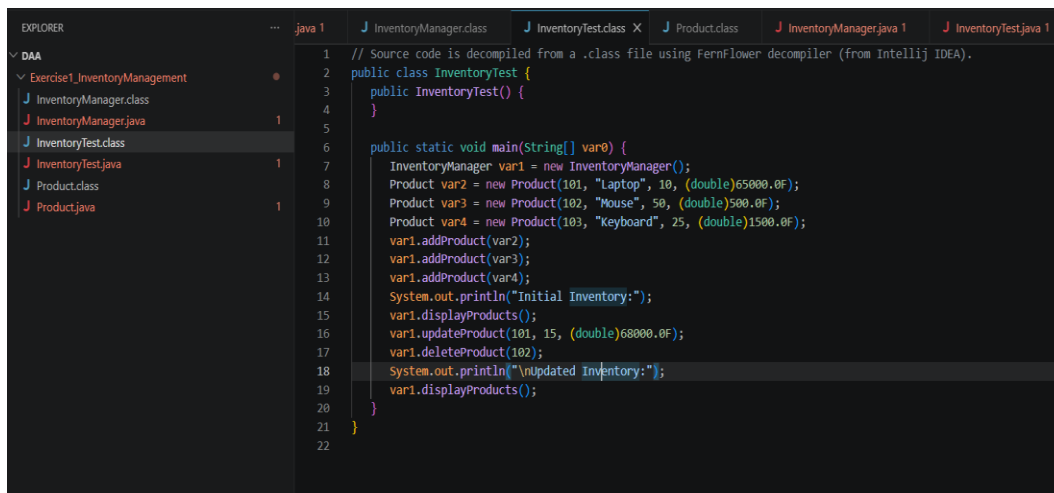


This screenshot shows the IntelliJ IDEA interface with the 'Product.class' file selected in the Explorer. The code editor displays the decompiled source code for the Product class. The Explorer on the left shows the project structure under 'DAA' > 'Exercise1_InventoryManagement', including 'InventoryManager.class', 'InventoryManager.java', 'InventoryTest.class', 'InventoryTest.java', 'Product.class', and 'Product.java'. The code editor shows the following code:

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 public class Product {
3     int productId;
4     String productName;
5     int quantity;
6     double price;
7
8     public Product(int var1, String var2, int var3, double var4) {
9         this.productId = var1;
10        this.productName = var2;
11        this.quantity = var3;
12        this.price = var4;
13    }
14
15    public String toString() {
16        return "ID: " + this.productId + ", Name: " + this.productName + ", Quantity: " + this.quantity + ", Price: " + this.price;
17    }
18 }
19
```



This screenshot is identical to the previous one, showing the 'Product.class' file and its decompiled source code in the IntelliJ IDEA editor. The Explorer on the left shows the same project structure.



This screenshot shows the IntelliJ IDEA interface with the 'InventoryTest.class' file selected in the Explorer. The code editor displays the decompiled source code for the InventoryTest class. The Explorer on the left shows the project structure under 'DAA' > 'Exercise1_InventoryManagement', including 'InventoryManager.class', 'InventoryManager.java', 'InventoryTest.class', 'InventoryTest.java', 'Product.class', and 'Product.java'. The code editor shows the following code:

```
1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA).
2 public class InventoryTest {
3     public InventoryTest() {
4     }
5
6     public static void main(String[] var0) {
7         InventoryManager var1 = new InventoryManager();
8         Product var2 = new Product(101, "Laptop", 10, (double)65000.0F);
9         Product var3 = new Product(102, "Mouse", 50, (double)500.0F);
10        Product var4 = new Product(103, "Keyboard", 25, (double)1500.0F);
11        var1.addProduct(var2);
12        var1.addProduct(var3);
13        var1.addProduct(var4);
14        System.out.println("Initial Inventory:");
15        var1.displayProducts();
16        var1.updateProduct(101, 15, (double)68000.0F);
17        var1.deleteProduct(102);
18        System.out.println("Updated Inventory:");
19        var1.displayProducts();
20    }
21 }
22
```

RESULT :

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS Code - Exercise1_Inve

• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise1_InventoryManagement> javac *.java
• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise1_InventoryManagement> java InventoryTest
Initial Inventory:
ID: 101, Name: Laptop, Quantity: 10, Price: 65000.0
ID: 102, Name: Mouse, Quantity: 50, Price: 500.0
ID: 103, Name: Keyboard, Quantity: 25, Price: 1500.0
Product updated successfully.
Product deleted successfully.

Updated Inventory:
ID: 101, Name: Laptop, Quantity: 15, Price: 68000.0
ID: 103, Name: Keyboard, Quantity: 25, Price: 1500.0
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise1_InventoryManagement> 
```

Exercise 2: E-commerce Platform Search Function

```
EXPLORER
DAA
  Exercise 2-E-commerce Platform Search Function
    Product.class
    product.java 1
    SearchDemo.class
    SearchDemo.java 3
    SearchEngine.class
    SearchEngine.java
  Exercise1_InventoryManagement

product.java 1 X SearchEngine.java SearchDemo.java 3
Exercise 2-E-commerce Platform Search Function > product.java > Product

1
2
3 public class Product {
4
5     private int productId;
6     private String productName;
7     private String category;
8
9     public Product(int productId,
10                    String productName,
11                    String category) {
12
13         this.productId = productId;
14         this.productName = productName;
15         this.category = category;
16     }
17
18     public int getProductId() {
19         return productId;
20     }
21
22     public String getProductName() {
23         return productName;
24     }
25
26     public String getCategory() {
27         return category;
28     }
29 }
```

```
import java.util.Comparator;

public class SearchEngine {

    public static int performLinearSearch(
        Product[] products,
        String target) {

        for (int i = 0; i < products.length; i++) {

            if (products[i]
                .getProductName()
                .equalsIgnoreCase(target)) {

                return i;
            }
        }

        return -1;
    }

    public static void sortProducts(
        Product[] products) {

        Arrays.sort(
            products,
            Comparator.comparing(
                Product::getProductName,
                String.CASE_INSENSITIVE_ORDER));
    }
}
```

```
public static int performBinarySearch(
    Product[] products,
    String target) {

    int left = 0;
    int right = products.length - 1;

    while (left <= right) {

        int mid =
            left + (right - left) / 2;

        int result =
            products[mid]
                .getProductName()
                .compareToIgnoreCase(target);

        if (result == 0) {
            return mid;
        }

        if (result < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

```
Exercise 2-E-commerce Platform Search Function > J SearchDemo.java > SearchDemo > main(String[])
3 public class SearchDemo {
4
5     Run | Debug
6     public static void main(String[] args) {
7         Product[] products = new Product[] {
8             new Product(productId: 1, productName: "Laptop", category: "Electronics"),
9             new Product(productId: 2, productName: "Mouse", category: "Electronics"),
10            new Product(productId: 3, productName: "Keyboard", category: "Electronics"),
11            new Product(productId: 4, productName: "Smartphone", category: "Electronics"),
12            new Product(productId: 5, productName: "Headphones", category: "Electronics")
13        };
14
15        String target = "mouse";
16
17        System.out.println(x: "Linear Search (unsorted):");
18        int idx1 = SearchEngine.performLinearSearch(products, target);
19        System.out.println("Target: " + target + " -> index = " + idx1);
20
21        // Binary search requires sorted array
22        System.out.println(x: "\nSorting products by name...");
23        SearchEngine.sortProducts(products);
24
25        System.out.println(x: "Binary Search (sorted):");
26        int idx2 = SearchEngine.performBinarySearch(products, target);
27        System.out.println("Target: " + target + " -> index = " + idx2);
28
29        if (idx2 != -1) {
30            System.out.println("Found product: " + products[idx2].getProductName()
31                               + " (category=" + products[idx2].getCategory() + ")");
32        }
33    }
34 }
```

RESULT:

```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> cd C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function; javac Product.java SearchEngine.java SearchDemo.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function>
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> java SearchDemo
Linear Search (unsorted):
Target: mouse -> index = 1

Sorting products by name...
Binary Search (sorted):
Target: mouse -> index = 3
Found product: Mouse (category=Electronics)
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> 
```

Exercise 3: Sorting Customer Orders.

```
J CustomerOrder.java X J SortingUtility.java J SortingDemo.java
Exercise3_SortingCustomerOrders > J CustomerOrder.java > ...
1  package Exercise3_SortingCustomerOrders;
2
3
4  public class CustomerOrder {
5
6      private int orderId;
7      private String customerName;
8      private double totalPrice;
9
10     public CustomerOrder(int orderId,
11                          String customerName,
12                          double totalPrice) {
13
14         this.orderId = orderId;
15         this.customerName = customerName;
16         this.totalPrice = totalPrice;
17     }
18
19     public int getOrderId() {
20         return orderId;
21     }
22
23     public String getCustomerName() {
24         return customerName;
25     }
26
27     public double getTotalPrice() {
28         return totalPrice;
29     }
30
31     @Override
32     public String toString() {
33
34         return orderId +
35             " | " +
36             customerName +
37             " | $" +
38             totalPrice;
39     }
40 }
```

```
package Exercise3_SortingCustomerOrders;
```



```
public class SortingUtility {
```

```
    // Bubble Sort
```

```
    public static void bubbleSort(  
        CustomerOrder[] orders) {
```

```
        int n = orders.length;
```

```
        for (int i = 0; i < n - 1; i++) {
```

```
            for (int j = 0;  
                j < n - i - 1;  
                j++) {
```

```
                if (orders[j].getTotalPrice()  
                    >  
                    orders[j + 1].getTotalPrice()) {
```

```
                    CustomerOrder temp =  
                        orders[j];
```

```
                    orders[j] =  
                        orders[j + 1];
```

```
                    orders[j + 1] =  
                        temp;
```

```
                }
```

```
            }
```

```
        }
```

```
    // Quick Sort
```

```
    public static void quickSort(  
        CustomerOrder[] orders,  
        int low,  
        int high) {
```

```
        if (low < high) {
```

```
            int pivotIndex =  
                partition(  
                    orders,  
                    low,  
                    high);
```

```
            quickSort(  
                orders,  
                low,  
                pivotIndex - 1);
```

```
            quickSort(  
                orders,  
                pivotIndex + 1,  
                high);
```

```
        }
```

```
    }
```

```
    private static int partition(  
        CustomerOrder[] orders,  
        int low,  
        int high) {
```

CustomerOrder.java J SortingUtility.java X J SortingDemo.java

Exercise3_SortingCustomerOrders > J SortingUtility.java > {} Exercise3_SortingCustomerOrders

```

3   public class SortingUtility {
35      public static void quickSort(
49          orders,
50          low,
51          pivotIndex - 1);
52
53          quickSort(
54              orders,
55              pivotIndex + 1,
56              high);
57      }
58  }
59
60      private static int partition(
61          CustomerOrder[] orders,
62          int low,
63          int high) {
64
65          double pivot =
66              orders[high]
67                  .getTotalPrice();
68
69          int i = low - 1;
70
71          for (int j = low;
72              j < high;
73              j++) {
74
75              if (orders[j]
76                  .getTotalPrice()
77                  < pivot) {
78
79                  i++;
80
81                  CustomerOrder temp =
82                      orders[i];
83
84                  orders[i] =
85                      orders[j];
86
87                  orders[j] =
88                      temp;
89              }
90          }
91
92          CustomerOrder temp =
93              orders[i + 1];
94
95          orders[i + 1] =
96              orders[high];
97
98          orders[high] =
99              temp;
100
101          return i + 1;
102      }
103  }
104

```

```

1 package Exercise3_SortingCustomerOrders;
2
3 public class SortingDemo {
4
5     public static void displayOrders(
6         CustomerOrder[] orders) {
7
8         for (CustomerOrder order
9             : orders) {
10
11             System.out.println(order);
12         }
13     }
14
15     Run | Debug
16     public static void main(String[] args) {
17
18         CustomerOrder[] bubbleOrders = {
19
20             new CustomerOrder(
21                 orderId: 301,
22                 customerName: "Deepak",
23                 totalPrice: 4200),
24
25             new CustomerOrder(
26                 orderId: 302,
27                 customerName: "Rahul",
28                 totalPrice: 1500),
29
30             new CustomerOrder(
31                 orderId: 303,
32                 customerName: "Kiran",
33                 totalPrice: 8700),
34
35             new CustomerOrder(
36                 orderId: 304,
37                 customerName: "Ananya",
38                 totalPrice: 2500)
39         };
40
41         System.out.println(
42             x: "Before Bubble Sort:");
43
44         displayOrders(
45             bubbleOrders);
46
47         SortingUtility.bubbleSort(
48             bubbleOrders);
49
50         System.out.println(
51             x: "\nAfter Bubble Sort:");

```



```
J CustomerOrder.java  J SortingUtility.java  J SortingDemo.java X
Exercise3_SortingCustomerOrders > J SortingDemo.java > {} Exercise3_SortingCustomerOrders
3  public class SortingDemo {
15      public static void main(String[] args) {
48
49          System.out.println(
50              x: "\nAfter Bubble Sort:");
51
52          displayOrders(
53              bubbleOrders);
54
55          CustomerOrder[] quickOrders = {
56
57              new CustomerOrder(
58                  orderId: 301,
59                  customerName: "Deepak",
60                  totalPrice: 4200),
61
62              new CustomerOrder(
63                  orderId: 302,
64                  customerName: "Rahul",
65                  totalPrice: 1500),
66
67              new CustomerOrder(
68                  orderId: 303,
69                  customerName: "Kiran",
70                  totalPrice: 8700),
71
72              new CustomerOrder(
73                  orderId: 304,
74                  customerName: "Ananya",
75                  totalPrice: 2500)
76          };
77
78          SortingUtility.quickSort(
79              quickOrders,
80              low: 0,
81              quickOrders.length - 1);
82
83          System.out.println(
84              x: "\nAfter Quick Sort:");
85
86          displayOrders(
87              quickOrders);
88      }
89  }
90
```

RESULT :

```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise3_SortingCustomerOrders> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise3_SortingCustomerOrders> java Sorting

Before Bubble Sort:
301 | Deepak | $4200.0
302 | Rahul | $1500.0
303 | Kiran | $8700.0
304 | Ananya | $2500.0

After Bubble Sort:
302 | Rahul | $1500.0
304 | Ananya | $2500.0
301 | Deepak | $4200.0
303 | Kiran | $8700.0

After Quick Sort:
302 | Rahul | $1500.0
304 | Ananya | $2500.0
301 | Deepak | $4200.0
303 | Kiran | $8700.0
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise3_SortingCustomerOrders>
```

Exercise 4: Employee Management System

```
Employee.java 1 X EmployeeDirectory.java 1 EmployeeDemo.java 1
Exercise4_EmployeeManagement > Employee.java > ...
1
2 public class Employee {
3
4     private int employeeId;
5     private String name;
6     private String position;
7     private double salary;
8
9     public Employee(int employeeId,
10                    String name,
11                    String position,
12                    double salary) {
13
14         this.employeeId = employeeId;
15         this.name = name;
16         this.position = position;
17         this.salary = salary;
18     }
19
20     public int getEmployeeId() {
21         return employeeId;
22     }
23
24     public String getName() {
25         return name;
26     }
27
28     public String getPosition() {
29         return position;
30     }
31
32     public double getSalary() {
33         return salary;
34     }
35
36     @Override
37     public String toString() {
38
39         return employeeId +
40             " | " +
41             name +
42             " | " +
43             position +
44             " | $" +
45             salary;
46     }
47 }
48
```

Exercise4_EmployeeManagement > J EmployeeDirectory.java > ...

```
1
2
3 public class EmployeeDirectory {
4
5     private Employee[] employees;
6     private int count;
7
8     public EmployeeDirectory(int size) {
9
10         employees = new Employee[size];
11         count = 0;
12     }
13
14     public void addEmployee(Employee employee) {
15
16         if (count < employees.length) {
17
18             employees[count] = employee;
19             count++;
20         }
21         else {
22
23             System.out.println(
24                 | | x: "Employee storage is full.");
25         }
26     }
27
28     public Employee searchEmployee(int id) {
29
30         for (int i = 0; i < count; i++) {
31
32             if (employees[i]
33                 | | .getEmployeeId() == id) {
34
35                 return employees[i];
36             }
37         }
38
39         return null;
40     }
41
42     public void displayEmployees() {
43
44         for (int i = 0; i < count; i++) {
45
46             System.out.println(
47                 | | employees[i]);
48         }
49     }
50
51     public void deleteEmployee(int id) {
```

```
41
42 public void displayEmployees() {
43
44     for (int i = 0; i < count; i++) {
45
46         System.out.println(
47             employees[i]);
48     }
49 }
50
51 public void deleteEmployee(int id) {
52
53     for (int i = 0; i < count; i++) {
54
55         if (employees[i]
56             .getEmployeeId() == id) {
57
58             for (int j = i;
59                 j < count - 1;
60                 j++) {
61
62                 employees[j] =
63                     employees[j + 1];
64             }
65
66             employees[count - 1] = null;
67             count--;
68
69             System.out.println(
70                 x: "Employee removed.");
71
72             return;
73         }
74     }
75
76     System.out.println(
77         x: "Employee not found.");
78 }
79 }
80
```

Exercise4_EmployeeManagement > J EmployeeDemo.java > ...

```
1
2 public class EmployeeDemo {
3
4     Run | Debug
5     public static void main(String[] args) {
6
7         EmployeeDirectory directory =
8             new EmployeeDirectory(size: 10);
9
10        directory.addEmployee(
11            new Employee(
12                employeeId: 101,
13                name: "Deepak",
14                position: "Software Engineer",
15                salary: 65000));
16
17        directory.addEmployee(
18            new Employee(
19                employeeId: 102,
20                name: "Rahul",
21                position: "QA Engineer",
22                salary: 55000));
23
24        directory.addEmployee(
25            new Employee(
26                employeeId: 103,
27                name: "Ananya",
28                position: "Business Analyst",
29                salary: 60000));
30
31        System.out.println(
32            x: "All Employees:");
33        directory.displayEmployees();
34
35        System.out.println(
36            x: "\nSearching Employee 102:");
37
38        Employee employee =
39            directory.searchEmployee(id: 102);
40
41        if (employee != null) {
42
43            System.out.println(
44                employee);
45        }
46
47        System.out.println(
48            x: "\nDeleting Employee 101:");
49
50        directory.deleteEmployee(id: 101);
51
52        System.out.println(
53            x: "\nUpdated Employee List:");
54
55        directory.displayEmployees();
56    }
57 }
58
```

RESULT :

```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise4_EmployeeManagement> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise4_EmployeeManagement> java EmployeeDemo

All Employees:
101 | Deepak | Software Engineer | $65000.0
102 | Rahul | QA Engineer | $55000.0
103 | Ananya | Business Analyst | $60000.0

Searching Employee 102:
102 | Rahul | QA Engineer | $55000.0

Deleting Employee 101:
Employee removed.

Updated Employee List:
102 | Rahul | QA Engineer | $55000.0
103 | Ananya | Business Analyst | $60000.0
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise4_EmployeeManagement>
```

Exercise 5: Task Management System

```
J Task.java 1 x J TaskList.java 1 J TaskDemo.java 1
Exercise5_TaskManagement > J Task.java > Task
1 public class Task {
2
3     int taskId;
4     String taskName;
5     String status;
6     Task next;
7
8     public Task(int taskId,
9                 String taskName,
10                String status) {
11
12         this.taskId = taskId;
13         this.taskName = taskName;
14         this.status = status;
15         this.next = null;
16     }
17 }
```

J Task.java 1 J TaskList.java 1 X J TaskDemo.java 1

Exercise5_TaskManagement > J TaskList.java > TaskList

```
1 public class TaskList {
2
3     Task head;
4
5     public void addTask(int id,
6                         String name,
7                         String status) {
8
9         Task newTask =
10             new Task(id, name, status);
11
12         if (head == null) {
13             head = newTask;
14             return;
15         }
16
17         Task temp = head;
18
19         while (temp.next != null) {
20             temp = temp.next;
21         }
22
23         temp.next = newTask;
24     }
25
26     public void showTasks() {
27
28         Task temp = head;
29
30         while (temp != null) {
31
32             System.out.println(
33                 temp.taskId + " | " +
34                 temp.taskName + " | " +
35                 temp.status);
36
37             temp = temp.next;
38         }
39     }
40 }
```



```
public Task findTask(int id) {  
    Task temp = head;  
  
    while (temp != null) {  
        if (temp.taskId == id) {  
            return temp;  
        }  
  
        temp = temp.next;  
    }  
  
    return null;  
}  
  
public void removeTask(int id) {  
    if (head == null) {  
        return;  
    }  
  
    if (head.taskId == id) {  
        head = head.next;  
        return;  
    }  
  
    Task current = head;  
  
    while (current.next != null) {  
        if (current.next.taskId == id) {  
            current.next =  
                current.next.next;  
            return;  
        }  
  
        current = current.next;  
    }  
}
```

Exercise5_TaskManagement > J TaskDemo.java > TaskDemo

```
1 public class TaskDemo {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         TaskList tasks =
7             new TaskList();
8
9         tasks.addTask(
10             id: 101,
11             name: "Create UI",
12             status: "Pending");
13
14         tasks.addTask(
15             id: 102,
16             name: "Develop Backend",
17             status: "In Progress");
18
19         tasks.addTask(
20             id: 103,
21             name: "Testing",
22             status: "Completed");
23
24         System.out.println(
25             x: "All Tasks:");
26
27         tasks.showTasks();
28
29         System.out.println(
30             x: "\nSearching Task 102:");
31
32         Task task =
33             tasks.findTask(id: 102);
34
35         if (task != null) {
36
37             System.out.println(
38                 "Found: "
39                 + task.taskName);
40
41             System.out.println(
42                 x: "\nDeleting Task 101");
43
44             tasks.removeTask(id: 101);
45
46             System.out.println(
47                 x: "\nRemaining Tasks:");
48
49             tasks.showTasks();
50         }
```

RESULT :

```
PROBLEMS 24 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise5_TaskManagement> java TaskDemo
All Tasks:
101 | Create UI | Pending
102 | Develop Backend | In Progress
103 | Testing | Completed

Searching Task 102:
Found: Develop Backend

Deleting Task 101

Remaining Tasks:
102 | Develop Backend | In Progress
103 | Testing | Completed
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise5_TaskManagement> 
```

Exercise 6: Library Management System

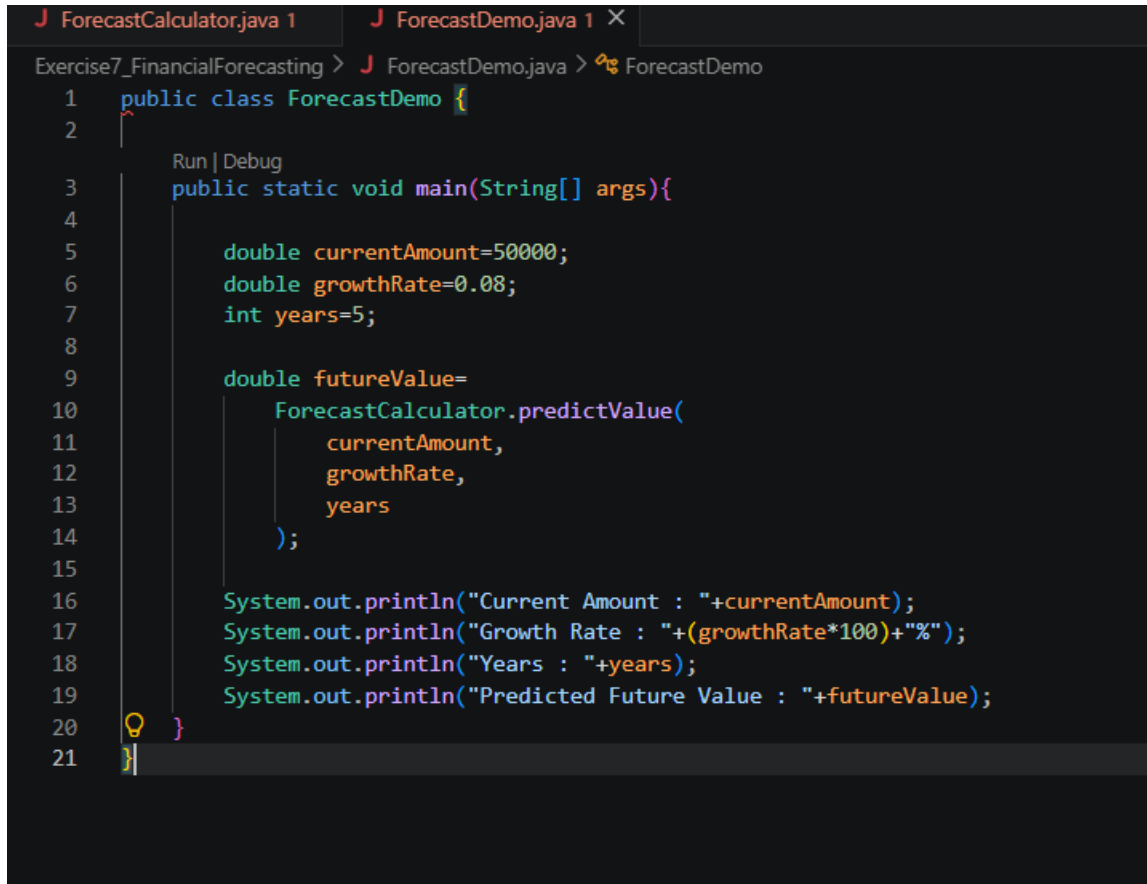
```
J LibraryDemo.java 1 X J Book.java 1
Exercise6_LibraryManagement > J LibraryDemo.java > LibraryDemo
1  import java.util.Arrays;
2  import java.util.Comparator;
3
4  public class LibraryDemo {
5
6      static Book linearSearch(Book[] books,String title){
7          for(Book b:books){
8              if(b.title.equalsIgnoreCase(title))
9                  return b;
10         }
11         return null;
12     }
13
14     static Book binarySearch(Book[] books,String title){
15         Arrays.sort(books,Comparator.comparing(b->b.title));
16
17         int low=0,high=books.length-1;
18
19         while(low<=high){
20             int mid=(low+high)/2;
21             int cmp=title.compareToIgnoreCase(books[mid].title);
22
23             if(cmp==0) return books[mid];
24             if(cmp>0) low=mid+1;
25             else high=mid-1;
26         }
27         return null;
28     }
29
30     Run | Debug
31     public static void main(String[] args){
32
33         Book[] books={
34             new Book(bookId: 101,title: "Operating Systems",author: "Galvin"),
35             new Book(bookId: 102,title: "Computer Networks",author: "Tanenbaum"),
36             new Book(bookId: 103,title: "Database Systems",author: "Korth"),
37             new Book(bookId: 104,title: "Java Programming",author: "Herbert Schildt")
38         };
39
40         Book b1=linearSearch(books,title: "Database Systems");
41
42         if(b1!=null)
43             System.out.println("Linear Search Result: "+b1.title);
44
45         Book b2=binarySearch(books,title: "Java Programming");
46
47         if(b2!=null)
48             System.out.println("Binary Search Result: "+b2.title);
49     }
```

```
J LibraryDemo.java 1 J Book.java 1 X
Exercise6_LibraryManagement > J Book.java > ...
1
2 public class Book {
3     int bookId;
4     String title;
5     String author;
6
7     public Book(int bookId,String title,String author){
8         this.bookId=bookId;
9         this.title=title;
10        this.author=author;
11    }
12 }
```

RESULT :

```
PROBLEMS 26 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise6_LibraryManagement> java LibraryDemo
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise6_LibraryManagement> java LibraryDemo
Linear Search Result: Database Systems
Binary Search Result: Java Programming
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise6_LibraryManagement> |
```

Exercise 7: Financial Forecasting



```
ForecastCalculator.java 1  ForecastDemo.java 1 X
Exercise7_FinancialForecasting > ForecastDemo.java > ForecastDemo
1 public class ForecastDemo {
2
3     Run | Debug
4     public static void main(String[] args){
5
6         double currentAmount=50000;
7         double growthRate=0.08;
8         int years=5;
9
10        double futureValue=
11            ForecastCalculator.predictValue(
12                currentAmount,
13                growthRate,
14                years
15            );
16
17        System.out.println("Current Amount : "+currentAmount);
18        System.out.println("Growth Rate : "+(growthRate*100)+"%");
19        System.out.println("Years : "+years);
20        System.out.println("Predicted Future Value : "+futureValue);
21    }
```

```
J ForecastCalculator.java 1 X J ForecastDemo.java 1
Exercise7_FinancialForecasting > J ForecastCalculator.java > ForecastCalculator
1 public class ForecastCalculator {
2
3     public static double predictValue(double amount,double growthRate,int years){
4         if(years==0)
5             return amount;
6
7         return predictValue(amount*(1+growthRate),growthRate,years-1);
8     }
9 }
```

RESULT :

```
PROBLEMS 28 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> javac *.java
● PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> java ForecastDemo
Current Amount : 50000.0
Growth Rate : 8.0%
Years : 5
Predicted Future Value : 73466.40384000003
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> 
```